

HockeyTrack Scoreboard — design

2026-09-06. Status: draft for review.

A physical LED scoreboard that follows a live NHL game, driven entirely by HockeyTrack's EventBridge bus. It is a separate project from HockeyTrack and is meant to double as the worked example of "one EventBridge rule away": nothing in the pipeline changes to support it beyond two small additions to the event contract (§6).

1. Goals

- A 4:1 LED panel (128×32 pixels, ~12" wide) shows, for one chosen game: game clock, period, score, shots on goal, the players currently in the penalty box with their penalty clocks, power-play state, and a goal flash.
- Updates reach the panel by push, within about a second of HockeyTrack publishing the event. No polling by the device.
- The device recovers from Wi-Fi drops, reboots and power cuts without help: on reconnect it shows the correct current state immediately.
- The owner can pick which game the board follows.
- The whole thing costs cents per month to run.

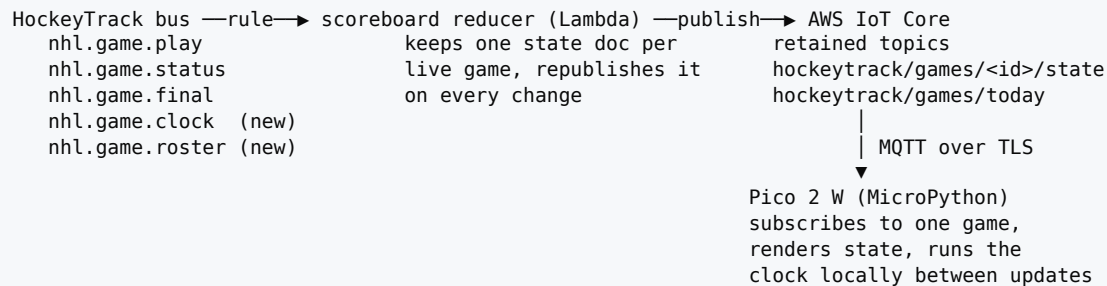
Non-goals for v1: audio, multiple simultaneous games on one panel, anything that requires the device to talk to the NHL directly.

2. Hardware

Part	Choice	Why
Controller	Raspberry Pi Pico 2 W (RP2350, Wi-Fi)	Dual-core, 520 KB RAM: enough for TLS + MQTT + a 128×32 frame buffer
Carrier	Pimoroni Interstate 75 W (Pico 2 W edition)	Drives HUB75 panels from PIO, exposes two user buttons, has a MicroPython graphics library; avoids hand-writing a panel driver
Panel	2 × 64×32 HUB75 LED matrix, chained	128×32 is exactly 4:1; at P2.5 pitch the pair is 320 × 80 mm (12.6" × 3.1")
Power	5 V / 4 A	Two panels at full white draw ~3.5 A

Any other 4:1 panel works if the driver exposes a 128×32 (or scaled) frame buffer; the firmware's layout is expressed in a 128×32 logical grid.

3. Architecture



Everything lives in the HockeyTrack AWS account (decision 1) but in its own Terraform stack in this repo. HockeyTrack knows nothing about it.

3.1 Transport: MQTT via AWS IoT Core

Chosen over a WebSocket API and over HTTP polling because:

- **Retained messages** make reconnects trivial: the broker hands a newly connected device the last state document at once. With an event stream the device would have to rebuild the game from history.
- Keepalive, reconnect, QoS and last-will are protocol features rather than code on the device. (API Gateway WebSockets also force a reconnect every two hours.)
- MicroPython's `umqtt` client with `ssl` and a per-device X.509 certificate runs on the Pico W within memory.
- Cost is effectively zero: IoT Core charges per million connection-minutes and messages; one device online all season is a few cents.

3.2 State, not events

The device is a pure renderer. The reducer folds bus events into one compact JSON document per game (≈300–500 bytes) and republishes it, retained, on every change:

```
{
  "v": 1, "gameId": 2026020001, "state": "LIVE",
  "asOf": 1791135723123,
  "away": {"abbrev": "TBL", "score": 2, "sog": 17, "color": "002868"},
  "home": {"abbrev": "NYR", "score": 1, "sog": 22, "color": "0038A8"},
  "period": {"number": 2, "type": "REG", "label": "2"},
  "clock": {"seconds": 872, "running": true, "intermission": false},
  "situation": {"code": "1451", "pp": "TBL", "emptyNet": null},
  "penalties": [
    {"team": "NYR", "number": 23, "seconds": 74, "type": "MIN", "endsOnGoal": true}
  ],
  "lastGoal": {"team": "TBL", "number": 86, "asOf": 1791135690000},
  "start": "2026-10-01T23:30:00Z"
}
```

`asOf` is the reducer's publish time. Clock and penalty seconds are as of that instant; the device counts them down locally when `running` is true and re-syncs on every message, so the display ticks every second even though HockeyTrack samples the feed every five.

A second retained topic, `hockeytrack/games/today`, lists the day's games (`gameId`, `away`, `home`, `start`, `state`) so the device can offer a choice without any server-side per-device state.

3.3 Reducer (Lambda, Go)

One function, one EventBridge rule matching the five detail-types above. Per game it keeps the state document in DynamoDB (single small table, TTL 48 h) so cold starts and concurrent invocations converge; it publishes with `iot:Publish` after each fold. Folds:

- `status` → state, period, score; PRE → LIVE also clears penalties.
- `clock` (new) → clock seconds/running/intermission, period, shots, situation code. This is the heartbeat; it arrives every 5 s while live.
- `play goal` → score, lastGoal (number via roster), and for the shorthanded side's *minor* penalties: end the oldest one (`endsOnGoal`).
- `play penalty` → append {team, number, seconds: duration×60, type}; bench/goalie minors use `servedByPlayerId`. Penalty timers run off the game clock: on each heartbeat, remaining = original – elapsed game time.
- `play period-start / period-end` → period label (1, 2, 3, OT, SO), intermission.
- `roster` (new) → `playerId` → sweater number map, kept in the same item.
- `final` → state FINAL; document stays retained until the day topic rolls.

Situation code (1451 = away goalie, away skaters, home skaters, home goalie) is the authority for `pp` and `emptyNet`; the penalty list is the detail beneath it. When the two disagree (delayed penalty, offsetting minors) the situation code wins for the PP indicator.

Cost: a heartbeat every 5 s per live game ≈ 720 invocations per game-hour; ≈ 3 M invocations a season ≈ \$1 with duration. DynamoDB and IoT are cents.

3.4 Device firmware (MicroPython)

- Boot: join Wi-Fi (credentials in a `secrets.py` never committed), NTP sync, load the last chosen `gameId` from flash, connect to IoT Core with the device certificate, subscribe to `hockeytrack/games/today` and `hockeytrack/games/<gameId>/state`.
- Render loop at 10 Hz: draw from the last state doc; derive displayed clock and penalty clocks as `seconds – (now – asOf)` while running.
- Buttons: A cycles through today's games (from the day topic), B toggles brightness; the choice is written to flash. Holding A during boot enters a Wi-Fi setup mode (v2; v1 uses `secrets.py`).
- Reconnect with backoff; while disconnected, keep rendering the last state with a small "no link" glyph.
- Pre-game (state: PRE): puck-drop countdown from `start`. Post-game: FINAL held with the final score.

3.5 Screen layout (128 × 32)

TBL	2	14:32	P2	1	NYR	← abbrevs in team colour
SOG	17	PP 1:14		SOG	22	
	NYR	23	1:14			← penalty rows (up to 2 per side)

Clock in a 7×13 digit font, everything else in 5×7. Goal: the scoring team's side flashes for 3 s with the scorer's number. Intermission: the clock area shows INT 12:40 . Empty net: EN replaces SOG on that side.

3.6 Game selection (decision 2)

v1: the device's A button cycles the day's games; the chosen id persists. v2: a small static web page in this repo (hosted alongside the site or on its own CloudFront) lists today's games and lets a signed-in owner publish {gameId} to scoreboard/<deviceId>/config , which the device also subscribes to. No per-device server state is needed for v1.

4. Security

- One IoT thing and certificate per device, provisioned by a script in tools/ ; the IoT policy allows Connect as its own client id and Subscribe/Receive on hockeytrack/games/* only. Devices publish nothing in v1.
- The reducer's role may publish only to hockeytrack/games/* , read/write only its own table, and write only its own log group.
- Everything the device renders is text from NHL data: abbreviations, numbers, times. The renderer takes only the typed fields and never interprets strings.
- Device secrets (secrets.py , certificate, key) are gitignored; the provisioning script prints exactly what to copy to the device.

5. Repository layout

hockeytrack-scoreboard/	
README.md	what it is, the wiring photo, the 3-step setup
firmware/	MicroPython: main.py, render.py, mqtt.py, fonts/
cloud/	Go: cmd/reducer, internal/reduce (pure fold + tests)
terraform/	rule on the hockeytrack bus, Lambda, DynamoDB, IoT policy
tools/provision.sh	create thing + cert, emit device files
docs/superpowers/	this spec, the plan

Tests: the fold is a pure function `reduce(state, event) → state` , tested with real HockeyTrack events captured from the bus (and, until the season starts, synthesised from the archived play-by-play via the replay harness). Firmware rendering is testable on a desktop MicroPython with a stub display.

6. Changes needed in HockeyTrack (filed as HOC tickets)

1. **nhl.game.clock heartbeat**. While a game is LIVE/CRIT, publish once per poll: gameId, period descriptor, clock (seconds remaining, running, intermission), both teams' score and shots, situation code. ≈720 events/game-hour, negligible cost; no change to existing detail-types.
2. **nhl.game.roster event** on the first poll of a game (and when the roster changes): gameId, and for each roster spot playerId, teamId, sweater number, position. Lets consumers print "#86" without an NHL call.

Both are additive and versioned under the existing `schemaVersion`.

7. Milestones

1. Platform: the two events above, deployed and visible on the bus.
2. Reducer + Terraform: state documents on the retained topics, verified with replayed games.
3. Firmware on the bench: renders a fixed state doc, then a live topic.
4. Game selection, reconnect handling, pre/post-game screens.
5. README as the worked example: "subscribe to HockeyTrack from another project in one rule."

8. Open questions

- Team colours: ship a 32-entry table in the reducer (hex), or omit and use a fixed palette? (Proposal: ship the table; it is small and the panel is the point.)
- Shootout display: one line of X/O per attempt, or just the score? (Proposal: score only in v1.)
- Should the `today` topic include yesterday's unfinished games (late West Coast starts cross midnight ET)? (Proposal: yes, games with state not FINAL from the previous day are included until noon ET.)